

ГУАП

КАФЕДРА № 44

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

старший преподаватель,
должность, уч. степень, звание

подпись, дата

Н.А. Соловьева
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №3

Стек и очередь

по курсу: Алгоритмы и структуры данных

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4341В

подпись, дата

Р.С. Кулаков
инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Изучение структур данных «стек» и «очередь», а также получение практических навыков их реализации.

2 Вариант задания

№ варианта	Номер задачи	Реализация
11	3	Стек – динамический; очередь – статическая

Задача 3

Система состоит из процессора P, трёх очередей F0, F1, F2 и стека S. В систему поступают запросы на выполнение задач.

Поступающие запросы ставятся в соответствующие приоритетам очереди. Сначала обрабатываются задачи из очереди F0. Если она пуста, можно обрабатывать задачи из очереди F1. Если и она пуста, то можно обрабатывать задачи из очереди F2. Если все очереди пусты, то система находится в ожидании поступающих задач (процессор свободен), либо в режиме обработки предыдущей задачи (процессор занят). Если поступает задача с более высоким приоритетом, чем обрабатываемая в данный момент, то обрабатываемая помещается в стек и может обрабатываться тогда и только тогда, когда все очереди пусты.

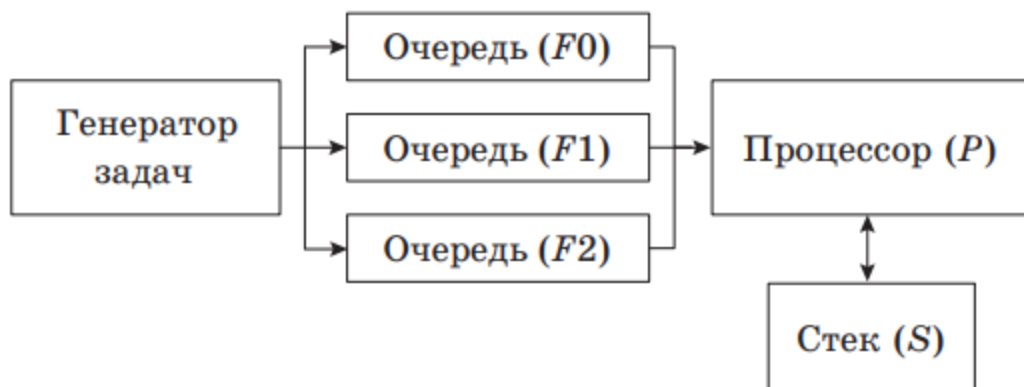


Рисунок 1 – Схема задачи 3

2.1 Правила работы системы

2.1.1 Поступающие задачи помещаются в очередь:

- F0 — высокий приоритет
- F1 — средний
- F2 — низкий

Обработка задач согласно приоритету: сначала F0, затем F1, затем F2

Если все очереди пусты: процессор либо свободен, либо завершает текущую задачу. Если приходит задача с более высоким приоритетом:

- текущая задача прерывается
- помещается в стек
- новая задача начинает выполняться

Задачи из стека выполняются только когда все очереди пусты

3 Листинг программы, реализующей поставленную задачу с использованием заданных структур данных

```
//
=====
====
//  Вариант 11 | Задача 3
//  Стек: динамический (связный список)
//  Очереди: статические (массив, кольцевой буфер)
//
//  Система: 1 процессор P, 3 очереди F0/F1/F2, 1 стек S.
//  Правило задачи 3: прерванная задача возвращается из
стека
//  ТОЛЬКО когда ВСЕ три очереди пусты.
//
=====
====

#include <iostream>
#include <string>
#include <vector>
```

```

#include <cstdlib>
#include <ctime>
#include <algorithm>

//
=====
====
// СТРУКТУРА «ЗАДАЧА»
//
=====
====
struct Task {
    std::string name;           // имя задачи (например "a",
    "b", ...)
    int priority;               // приоритет: 0 = высший, 1 =
    средний, 2 = низший
    int totalTime;             // полное время выполнения (в
    тактах)
    int remaining;             // оставшееся время выполнения
    int arrival;               // такт поступления в систему
};

//
=====
====
// СТАТИЧЕСКАЯ ОЧЕРЕДЬ
// Реализована через массив фиксированного размера
// (кольцевой буфер).
// head — откуда берём, tail — куда добавляем.
//
=====
====
const int QUEUE_CAP = 50; // максимальная ёмкость одной
очереди

struct StaticQueue {
    Task* buf[QUEUE_CAP]; // массив указателей на задачи
    int head;             // индекс первого элемента
    int tail;             // индекс за последним элементом
    int sz;               // текущее количество элементов
};

```

```

// Инициализация очереди
void initQueue(StaticQueue& q) {
    q.head = 0;
    q.tail = 0;
    q.sz    = 0;
}

// Проверки
bool qEmpty(const StaticQueue& q) { return q.sz == 0; }
bool qFull  (const StaticQueue& q) { return q.sz >=
QUEUE_CAP; }

// Добавить задачу в конец очереди (enqueue)
bool enqueue(StaticQueue& q, Task* t) {
    if (qFull(q)) {
        std::cout << "[OVERFLOW] Queue is full! Task '"
                    << t->name << "' rejected.\n";
        return false;
    }
    q.buf[q.tail] = t;
    q.tail = (q.tail + 1) % QUEUE_CAP; // кольцевой сдвиг
    q.sz++;
    return true;
}

// Извлечь задачу из начала очереди (dequeue)
Task* dequeue(StaticQueue& q) {
    if (qEmpty(q)) return nullptr;
    Task* t = q.buf[q.head];
    q.head  = (q.head + 1) % QUEUE_CAP;
    q.sz--;
    return t;
}

// Вывод содержимого очереди: начало слева, конец справа
void printQueue(const StaticQueue& q) {
    for (int i = 0; i < q.sz; i++) {
        std::cout << q.buf[(q.head + i) % QUEUE_CAP]->name;
        if (i < q.sz - 1) std::cout << " ";
    }
}

```

```

//
=====
====
// ДИНАМИЧЕСКИЙ СТЕК
// Реализован через односвязный список.
// Каждый новый узел выделяется в памяти через new.
//
=====
====
struct StackNode {
    Task*      task; // хранимая задача
    StackNode* next; // указатель на узел ниже в стеке
};

struct DynStack {
    StackNode* top; // вершина стека (nullptr если стек
пуст)
    int        sz;  // количество элементов
};

// Инициализация стека
void initStack(DynStack& s) {
    s.top = nullptr;
    s.sz  = 0;
}

// Проверка на пустоту
bool sEmpty(const DynStack& s) { return s.top == nullptr;
}

// Добавить задачу на вершину стека (push)
void push(DynStack& s, Task* t) {
    StackNode* node = new StackNode(); // динамическое
выделение памяти
    node->task = t;
    node->next = s.top; // новый узел смотрит на старую
вершину
    s.top      = node; // новый узел становится вершиной
    s.sz++;
}

// Извлечь задачу с вершины стека (pop)

```

```

Task* pop(DynStack& s) {
    if (s.Empty(s)) return nullptr;
    StackNode* node = s.top;
    Task* t = node->task;
    s.top = node->next;
    delete node; // освобождаем память узла
    s.sz--;
    return t;
}

// Вывод содержимого стека: вершина слева, дно справа
void printStack(const DynStack& s) {
    StackNode* cur = s.top;
    bool first = true;
    while (cur != nullptr) {
        if (!first) std::cout << " ";
        std::cout << cur->task->name;
        first = false;
        cur = cur->next;
    }
}

// Полная очистка стека (освобождение памяти)
void freeStack(DynStack& s) {
    while (!s.Empty(s)) {
        Task* t = pop(s);
        delete t;
    }
}

//
=====
====
// СИСТЕМА
//
=====
====
struct System {
    StaticQueue Q[3]; // Q[0]=F0 (высший приоритет),
Q[1]=F1, Q[2]=F2
    DynStack S; // стек S

```

```

    Task*          cpu;    // задача в процессоре (nullptr =
свободен)
    int            time;   // текущий такт
};

// Инициализация системы
void initSys(System& sys) {
    for (int i = 0; i < 3; i++) initQueue(sys.Q[i]);
    initStack(sys.S);
    sys.cpu = nullptr;
    sys.time = 0;
}

// Проверка: все три очереди пусты?
bool allEmpty(const System& sys) {
    for (int i = 0; i < 3; i++)
        if (!qEmpty(sys.Q[i])) return false;
    return true;
}

// Наивысший приоритет среди задач в очередях.
// Возвращает 0, 1 или 2. Возвращает -1 если все очереди
пусты.
int bestPrio(const System& sys) {
    for (int p = 0; p <= 2; p++)
        if (!qEmpty(sys.Q[p])) return p;
    return -1;
}

// Взять задачу из очереди с наивысшим приоритетом
Task* takeFromQueues(System& sys) {
    for (int p = 0; p <= 2; p++)
        if (!qEmpty(sys.Q[p])) return dequeue(sys.Q[p]);
    return nullptr;
}

// Вывод состояния системы (на текущем такте)
void printState(const System& sys) {
    std::cout << "Timer: " << sys.time;
    std::cout << "  F0:["; printQueue(sys.Q[0]); std::cout
<< "];";

```



```

        std::cout << "  F1:["; printQueue(sys.Q[1]); std::cout
<< "]"";
        std::cout << "  F2:["; printQueue(sys.Q[2]); std::cout
<< "]"";
        std::cout << "  CPU:";
        if (sys.cpu) std::cout << sys.cpu->name;
        else          std::cout << "-";
        std::cout << "  Stack:["; printStack(sys.S); std::cout
<< "]\n";
    }

    //
    =====
    ====
    //  ОДИН ТАКТ РАБОТЫ СИСТЕМЫ
    //
    //  Порядок событий внутри такта:
    //      1. Добавить поступившие задачи в очереди
    //      2. Проверить, нужно ли прервать текущую задачу
    //      3. Загрузить процессор (если свободен)
    //      4. Вывести состояние (ПЕРЕД выполнением — так
соответствует примеру)
    //      5. Выполнить один такт: уменьшить remaining
    //          Если задача завершена — удалить её (следующая
загрузится в след. такте)
    //
    =====
    ====
void tick(System& sys, std::vector<Task*>& arrivals) {

    // --- 1. Добавить поступившие задачи в очереди ---
    for (Task* t : arrivals) {
        // задача идёт в очередь, соответствующую её
приоритету
        enqueue(sys.Q[t->priority], t);
    }

    // --- 2. Проверить прерывание текущей задачи ---
    // Условие: в очереди есть задача с БОЛЕЕ высоким
приоритетом (меньший номер)
    if (sys.cpu != nullptr) {
        int bp = bestPrio(sys);

```

```

        if (bp != -1 && bp < sys.cpu->priority) {
            // Текущую задачу кладём в стек и освобождаем
процессор
            std::cout << "    [PREEMPT] Task '" << sys.cpu-
>name
                << "' (p=" << sys.cpu->priority
                << ") preempted by priority " << bp
<< " task.\n";
            push(sys.S, sys.cpu);
            sys.cpu = nullptr;
        }
    }

    // --- 3. Загрузить процессор если свободен ---
    if (sys.cpu == nullptr) {
        if (!allEmpty(sys)) {
            // Есть задачи в очередях — берём наиболее
приоритетную
            sys.cpu = takeFromQueues(sys);
        } else if (!sEmpty(sys.S)) {
            // ТОЛЬКО если все очереди пусты — берём из
стека
            // (ключевое правило задачи 3)
            sys.cpu = pop(sys.S);
        }
    }

    // --- 4. Вывод состояния (до выполнения такта) ---
    printState(sys);
    sys.time++;

    // --- 5. Выполнить один такт обработки ---
    if (sys.cpu != nullptr) {
        sys.cpu->remaining--;
        if (sys.cpu->remaining <= 0) {
            // Задача завершена
            std::cout << "                >> Task '" << sys.cpu-
>name
                << "' completed.\n";
            delete sys.cpu; // освобождаем память задачи
            sys.cpu = nullptr;

```

```

        // Следующая задача будет загружена в начале
        СЛЕДУЮЩЕГО такта
    }
}

//
=====
====
// ЗАПИСЬ В РАСПИСАНИИ (такт прихода + задача)
//
=====
====
struct Entry {
    int    when; // такт поступления
    Task* task; // указатель на задачу
};

//
=====
====
// ЗАПУСК СИМУЛЯЦИИ ПО РАСПИСАНИЮ
//
=====
====
void simulate(System& sys, std::vector<Entry>& sched) {
    // Подсчитываем верхнюю границу времени симуляции
    int latest = 0, totalWork = 0;
    for (auto& e : sched) {
        latest    = std::max(latest, e.when);
        totalWork += e.task->totalTime;
    }
    int endT = latest + totalWork + 5; // с небольшим
    запасом

    for (int t = 0; t < endT; t++) {
        // Собрать задачи, поступающие именно на такт t
        std::vector<Task*> arr;
        for (auto& e : sched) {
            if (e.when == t) arr.push_back(e.task);
        }
    }
}

```

```

        // Вывод сообщений о поступлении
        for (Task* tk : arr) {
            std::cout << "    --> Task '" << tk->name
                << "' arrives    priority=" << tk-
>priority
                << "    duration=" << tk->totalTime <<
"\n";
        }

        // Выполнить такт
        tick(sys, arr);

        // Ранний выход: нет будущих задач и система
простаивает
        bool future = false;
        for (auto& e : sched) {
            if (e.when > t) { future = true; break; }
        }
        if (!future && sys.cpu == nullptr && allEmpty(sys)
&& sEmpty(sys.S))
            break;
    }

    std::cout << "\n=== Simulation complete. All tasks
done. ===\n";
}

//
=====
====
//    РУЧНОЕ ДОБАВЛЕНИЕ ЗАДАЧИ
//
=====
====
void addManual(std::vector<Entry>& sched, int& counter) {
    Task* t = new Task();
    std::cout << "    Task name            : "; std::cin >> t-
>name;
    std::cout << "    Priority (0/1/2) : "; std::cin >> t-
>priority;
    std::cout << "    Duration (ticks) : "; std::cin >> t-
>totalTime;

```

```

        std::cout << "    Arrival time          : "; std::cin >> t-
>arrival;

        // Валидация
        if (t->priority < 0 || t->priority > 2) {
            std::cout << "    [ERROR] Priority must be 0, 1 or
2. Task rejected.\n";
            delete t; return;
        }
        if (t->totalTime <= 0) {
            std::cout << "    [ERROR] Duration must be > 0. Task
rejected.\n";
            delete t; return;
        }
        if (t->arrival < 0) {
            std::cout << "    [ERROR] Arrival time must be >= 0.
Task rejected.\n";
            delete t; return;
        }

        t->remaining = t->totalTime;
        sched.push_back({t->arrival, t});
        counter++;
        std::cout << "    Task '" << t->name << "' added
successfully.\n";
    }

    //
    =====
    ====
    // АВТОГЕНЕРАЦИЯ ЗАДАЧ
    //
    =====
    ====
    void autoGen(std::vector<Entry>& sched, int& counter) {
        int n;
        std::cout << "    How many tasks to generate? ";
        std::cin >> n;
        srand(static_cast<unsigned>(time(nullptr)));

        for (int i = 0; i < n; i++) {
            Task* t          = new Task();

```

```

        // Имена: A, B, C, ... (циклически)
        t->name      = std::string(1, char('A' + (counter
% 26)));
        t->priority  = rand() % 3;           // 0, 1 или 2
        t->totalTime = 1 + rand() % 8;      // от 1 до 8 тактов
        t->remaining = t->totalTime;
        t->arrival   = rand() % 15;         // от такта 0 до
14

        sched.push_back({t->arrival, t});
        counter++;

        std::cout << "  Generated: " << t->name
                    << "  priority=" << t->priority
                    << "  duration=" << t->totalTime
                    << "  arrival=" << t->arrival << "\n";
    }
}

//
=====
====
//  ВСТРОЕННЫЙ ПРИМЕР (адаптирован из методички)
//
=====
====
void builtinExample(std::vector<Entry>& sched) {
    // Вспомогательная лямбда для краткости
    auto add = [&](const std::string& nm, int prio, int
dur, int arr) {
        Task* t = new Task{nm, prio, dur, dur, arr};
        sched.push_back({arr, t});
    };
    //      имя   приор   длит   приход
    add("a",    2,      10,     2);
    add("b",    1,       5,     5);
    add("c",    0,       3,     7);
    add("d",    0,       5,    22);
    add("e",    1,       3,    22);
    add("f",    2,       5,    25);
}

```

```

//
=====
====
//  ГЛАВНАЯ ФУНКЦИЯ
//
=====
====
int main() {
    std::cout
"=====\\n";
    std::cout << " Variant 11 | Task 3\\n";
    std::cout << " Stack: DYNAMIC | Queues: STATIC\\n";
    std::cout << " Rule: stack task resumes ONLY when all
queues empty\\n";
    std::cout
"=====\\n\\n";

    int choice;
    std::cout << "Select mode:\\n";
    std::cout << "    1 - Built-in example (from the
manual)\\n";
    std::cout << "    2 - Enter tasks manually\\n";
    std::cout << "    3 - Auto-generate tasks\\n";
    std::cout << "Choice: ";
    std::cin >> choice;

    std::vector<Entry> sched;
    int counter = 0;

    if (choice == 1) {
        builtinExample(sched);
        std::cout << "\\nRunning built-in example...\\n\\n";
    }
    else if (choice == 2) {
        int n;
        std::cout << "How many tasks? ";
        std::cin >> n;
        for (int i = 0; i < n; i++) {
            std::cout << "\\n--- Task " << (i + 1) << " --
-\\n";
            addManual(sched, counter);
        }
    }
}

```

```

    }
    else if (choice == 3) {
        autoGen(sched, counter);
    }
    else {
        std::cout << "Invalid choice.\n";
        return 1;
    }

    // Создаём и запускаем систему
    System sys;
    initSys(sys);

    std::cout << "\n--- Simulation start ---\n\n";
    simulate(sys, sched);

    // Освобождаем память стека (если осталось что-то
    незавершённое)
    freeStack(sys.S);

    return 0;
}

```


4 Контрольные примеры, в которых задействованы все структуры описанные в задании

```
=====
Variant 11 | Task 3
Stack: DYNAMIC | Queues: STATIC
Rule: stack task resumes ONLY when all queues empty
=====

Select mode:
  1 - Built-in example (from the manual)
  2 - Enter tasks manually
  3 - Auto-generate tasks
Choice: 3
How many tasks to generate? 5
Generated: A priority=2 duration=7 arrival=10
Generated: B priority=2 duration=4 arrival=8
Generated: C priority=1 duration=2 arrival=3
Generated: D priority=1 duration=8 arrival=7
Generated: E priority=2 duration=4 arrival=3

--- Simulation start ---

Timer: 0 F0:[] F1:[] F2:[] CPU:- Stack:[]
Timer: 1 F0:[] F1:[] F2:[] CPU:- Stack:[]
Timer: 2 F0:[] F1:[] F2:[] CPU:- Stack:[]
    --> Task 'C' arrives priority=1 duration=2
    --> Task 'E' arrives priority=2 duration=4
Timer: 3 F0:[] F1:[] F2:[E] CPU:C Stack:[]
Timer: 4 F0:[] F1:[] F2:[E] CPU:C Stack:[]
    >> Task 'C' completed.
Timer: 5 F0:[] F1:[] F2:[] CPU:E Stack:[]
Timer: 6 F0:[] F1:[] F2:[] CPU:E Stack:[]
    --> Task 'D' arrives priority=1 duration=8
    [PREEMPT] Task 'E' (p=2) preempted by priority 1 task.
Timer: 7 F0:[] F1:[] F2:[] CPU:D Stack:[E]
    --> Task 'B' arrives priority=2 duration=4
Timer: 8 F0:[] F1:[] F2:[B] CPU:D Stack:[E]
Timer: 9 F0:[] F1:[] F2:[B] CPU:D Stack:[E]
    --> Task 'A' arrives priority=2 duration=7
Timer: 10 F0:[] F1:[] F2:[B A] CPU:D Stack:[E]
Timer: 11 F0:[] F1:[] F2:[B A] CPU:D Stack:[E]
Timer: 12 F0:[] F1:[] F2:[B A] CPU:D Stack:[E]
Timer: 13 F0:[] F1:[] F2:[B A] CPU:D Stack:[E]
Timer: 14 F0:[] F1:[] F2:[B A] CPU:D Stack:[E]
    >> Task 'D' completed.
Timer: 15 F0:[] F1:[] F2:[A] CPU:B Stack:[E]
Timer: 16 F0:[] F1:[] F2:[A] CPU:B Stack:[E]
Timer: 17 F0:[] F1:[] F2:[A] CPU:B Stack:[E]
Timer: 18 F0:[] F1:[] F2:[A] CPU:B Stack:[E]
    >> Task 'B' completed.
Timer: 19 F0:[] F1:[] F2:[] CPU:A Stack:[E]
Timer: 20 F0:[] F1:[] F2:[] CPU:A Stack:[E]
Timer: 21 F0:[] F1:[] F2:[] CPU:A Stack:[E]
Timer: 22 F0:[] F1:[] F2:[] CPU:A Stack:[E]
Timer: 23 F0:[] F1:[] F2:[] CPU:A Stack:[E]
Timer: 24 F0:[] F1:[] F2:[] CPU:A Stack:[E]
Timer: 25 F0:[] F1:[] F2:[] CPU:A Stack:[E]
    >> Task 'A' completed.
Timer: 26 F0:[] F1:[] F2:[] CPU:E Stack:[]
Timer: 27 F0:[] F1:[] F2:[] CPU:E Stack:[]
    >> Task 'E' completed.

=== Simulation complete. All tasks done. ===
```

Рисунок 2 – Пример работы 5 задач

```

--- Simulation start ---

Timer: 0  F0:[]  F1:[]  F2:[]  CPU:-  Stack:[]
Timer: 1  F0:[]  F1:[]  F2:[]  CPU:-  Stack:[]
--> Task 'a' arrives  priority=2  duration=10
Timer: 2  F0:[]  F1:[]  F2:[]  CPU:a  Stack:[]
Timer: 3  F0:[]  F1:[]  F2:[]  CPU:a  Stack:[]
Timer: 4  F0:[]  F1:[]  F2:[]  CPU:a  Stack:[]
--> Task 'b' arrives  priority=1  duration=5
[PREEMPT] Task 'a' (p=2) preempted by priority 1 task.
Timer: 5  F0:[]  F1:[]  F2:[]  CPU:b  Stack:[a]
Timer: 6  F0:[]  F1:[]  F2:[]  CPU:b  Stack:[a]
--> Task 'c' arrives  priority=0  duration=3
[PREEMPT] Task 'b' (p=1) preempted by priority 0 task.
Timer: 7  F0:[]  F1:[]  F2:[]  CPU:c  Stack:[b a]
Timer: 8  F0:[]  F1:[]  F2:[]  CPU:c  Stack:[b a]
Timer: 9  F0:[]  F1:[]  F2:[]  CPU:c  Stack:[b a]
>> Task 'c' completed.
Timer: 10 F0:[]  F1:[]  F2:[]  CPU:b  Stack:[a]
Timer: 11 F0:[]  F1:[]  F2:[]  CPU:b  Stack:[a]
Timer: 12 F0:[]  F1:[]  F2:[]  CPU:b  Stack:[a]
>> Task 'b' completed.
Timer: 13 F0:[]  F1:[]  F2:[]  CPU:a  Stack:[]
Timer: 14 F0:[]  F1:[]  F2:[]  CPU:a  Stack:[]
Timer: 15 F0:[]  F1:[]  F2:[]  CPU:a  Stack:[]
Timer: 16 F0:[]  F1:[]  F2:[]  CPU:a  Stack:[]
Timer: 17 F0:[]  F1:[]  F2:[]  CPU:a  Stack:[]
Timer: 18 F0:[]  F1:[]  F2:[]  CPU:a  Stack:[]
Timer: 19 F0:[]  F1:[]  F2:[]  CPU:a  Stack:[]
>> Task 'a' completed.
Timer: 20 F0:[]  F1:[]  F2:[]  CPU:-  Stack:[]
Timer: 21 F0:[]  F1:[]  F2:[]  CPU:-  Stack:[]
--> Task 'd' arrives  priority=0  duration=5
--> Task 'e' arrives  priority=1  duration=3
Timer: 22 F0:[]  F1:[e]  F2:[]  CPU:d  Stack:[]
Timer: 23 F0:[]  F1:[e]  F2:[]  CPU:d  Stack:[]
Timer: 24 F0:[]  F1:[e]  F2:[]  CPU:d  Stack:[]
--> Task 'f' arrives  priority=2  duration=5
Timer: 25 F0:[]  F1:[e]  F2:[f]  CPU:d  Stack:[]
Timer: 26 F0:[]  F1:[e]  F2:[f]  CPU:d  Stack:[]
>> Task 'd' completed.
Timer: 27 F0:[]  F1:[]  F2:[f]  CPU:e  Stack:[]
Timer: 28 F0:[]  F1:[]  F2:[f]  CPU:e  Stack:[]
Timer: 29 F0:[]  F1:[]  F2:[f]  CPU:e  Stack:[]
>> Task 'e' completed.
Timer: 30 F0:[]  F1:[]  F2:[]  CPU:f  Stack:[]
Timer: 31 F0:[]  F1:[]  F2:[]  CPU:f  Stack:[]
Timer: 32 F0:[]  F1:[]  F2:[]  CPU:f  Stack:[]
Timer: 33 F0:[]  F1:[]  F2:[]  CPU:f  Stack:[]
Timer: 34 F0:[]  F1:[]  F2:[]  CPU:f  Stack:[]
>> Task 'f' completed.

=== Simulation complete. All tasks done. ===

```

Рисунок 3 – Пример обработки 8 задач

5 Реализация на Kotlin

```
// =====  
// Variant 11 | Task 3 (Kotlin)  
// Stack: dynamic (linked list)  
// Queues: static (array, circular buffer)  
// =====  
  
import kotlin.random.Random  
  
// -----  
// TASK  
// -----  
data class Task(  
    var name: String,  
    var priority: Int,  
    var totalTime: Int,  
    var remaining: Int,  
    var arrival: Int  
)  
  
// -----  
// STATIC QUEUE  
// -----  
const val QUEUE_CAP = 50  
  
class StaticQueue {  
    private val buf = arrayOfNulls<Task>(QUEUE_CAP)  
    private var head = 0  
    private var tail = 0  
    private var size = 0  
  
    fun isEmpty() = size == 0
```

```

fun isFull() = size >= QUEUE_CAP

fun enqueue(t: Task): Boolean {
    if (isFull()) {
        println("[OVERFLOW] Queue full! Task '${t.name}'
rejected.")
        return false
    }
    buf[tail] = t
    tail = (tail + 1) % QUEUE_CAP
    size++
    return true
}

fun dequeue(): Task? {
    if (isEmpty()) return null
    val t = buf[head]
    head = (head + 1) % QUEUE_CAP
    size--
    return t
}

fun print() {
    for (i in 0 until size) {
        val t = buf[(head + i) % QUEUE_CAP]
        print(t?.name)
        if (i < size - 1) print(" ")
    }
}

// -----
// DYNAMIC STACK

```

```

// -----
class StackNode(
    var task: Task,
    var next: StackNode?
)

class DynStack {
    private var top: StackNode? = null

    fun isEmpty() = top == null

    fun push(t: Task) {
        top = StackNode(t, top)
    }

    fun pop(): Task? {
        val node = top ?: return null
        top = node.next
        return node.task
    }

    fun print() {
        var cur = top
        var first = true
        while (cur != null) {
            if (!first) print(" ")
            print(cur.task.name)
            first = false
            cur = cur.next
        }
    }
}

```

```

// -----
// SYSTEM
// -----

class System {
    val Q = Array(3) { StaticQueue() }
    val S = DynStack()
    var cpu: Task? = null
    var time = 0

    fun allQueuesEmpty(): Boolean {
        return Q.all { it.isEmpty() }
    }

    fun bestPriority(): Int {
        for (i in 0..2) {
            if (!Q[i].isEmpty()) return i
        }
        return -1
    }

    fun takeFromQueues(): Task? {
        for (i in 0..2) {
            if (!Q[i].isEmpty()) return Q[i].dequeue()
        }
        return null
    }

    fun printState() {
        print("Timer: $time ")
        print("F0:[" ); Q[0].print(); print("] ")
        print("F1:[" ); Q[1].print(); print("] ")
        print("F2:[" ); Q[2].print(); print("] ")
        print("CPU:")
    }
}

```

```

        print(cpu?.name ?: "-")
        print("  Stack:[" ); S.print(); print("]")
        println()
    }
}

// -----
// TICK
// -----
fun tick(sys: System, arrivals: List<Task>) {

    // 1. arrivals
    for (t in arrivals) {
        sys.Q[t.priority].enqueue(t)
    }

    // 2. preemption
    val cpu = sys.cpu
    if (cpu != null) {
        val bp = sys.bestPriority()
        if (bp != -1 && bp < cpu.priority) {
            println("  [PREEMPT] ${cpu.name}")
            sys.S.push(cpu)
            sys.cpu = null
        }
    }

    // 3. load CPU
    if (sys.cpu == null) {
        if (!sys.allQueuesEmpty()) {
            sys.cpu = sys.takeFromQueues()
        } else if (!sys.S.isEmpty()) {
            sys.cpu = sys.S.pop()
        }
    }
}

```

```

        }
    }

    // 4. print state
    sys.printState()
    sys.time++

    // 5. execute
    sys.cpu?.let {
        it.remaining--
        if (it.remaining <= 0) {
            println("  >> Task '${it.name}' completed")
            sys.cpu = null
        }
    }
}

// -----
// SIMULATION
// -----
data class Entry(val time: Int, val task: Task)

fun simulate(entries: List<Entry>) {
    val sys = System()

    val maxTime = entries.maxOf { it.time } +
        entries.sumOf { it.task.totalTime } + 5

    for (t in 0..maxTime) {
        val arrivals = entries.filter { it.time == t }.map {
it.task }

        for (a in arrivals) {

```



```

        println("  --> Task '${a.name}' arrives")
    }

    tick(sys, arrivals)

    val future = entries.any { it.time > t }
    if (!future && sys.cpu == null && sys.allQueuesEmpty() &&
sys.S.isEmpty()) {
        break
    }
}

println("\n=== Simulation complete ===")
}

// -----
// DEMO
// -----
fun demo(): List<Entry> {
    return listOf(
        Entry(2, Task("a", 2, 10, 10, 2)),
        Entry(5, Task("b", 1, 5, 5, 5)),
        Entry(7, Task("c", 0, 3, 3, 7)),
        Entry(22, Task("d", 0, 5, 5, 22)),
        Entry(22, Task("e", 1, 3, 3, 22)),
        Entry(25, Task("f", 2, 5, 5, 25))
    )
}

// -----
// MAIN
// -----
fun main() {

```

```
println("Variant 11 | Kotlin version\n")

val entries = demo()

println("--- Simulation start ---\n")
simulate(entries)
}
```

6 Выводы по работе

В ходе работы реализованы структуры данных стек и очередь без использования STL, также смоделирована система обработки задач с приоритетами. Реализован механизм вытеснения задач и возврата через стек, дополнительно продемонстрирована работа системы на контрольном примере